LAPORAN PRAKTIKUM

POSTTEST 5

ALGORITMA PEMROGRAMAN DASAR



Disusun oleh:

Chika Pramulistia Putri 2609106006

Kelas A1 ‘26

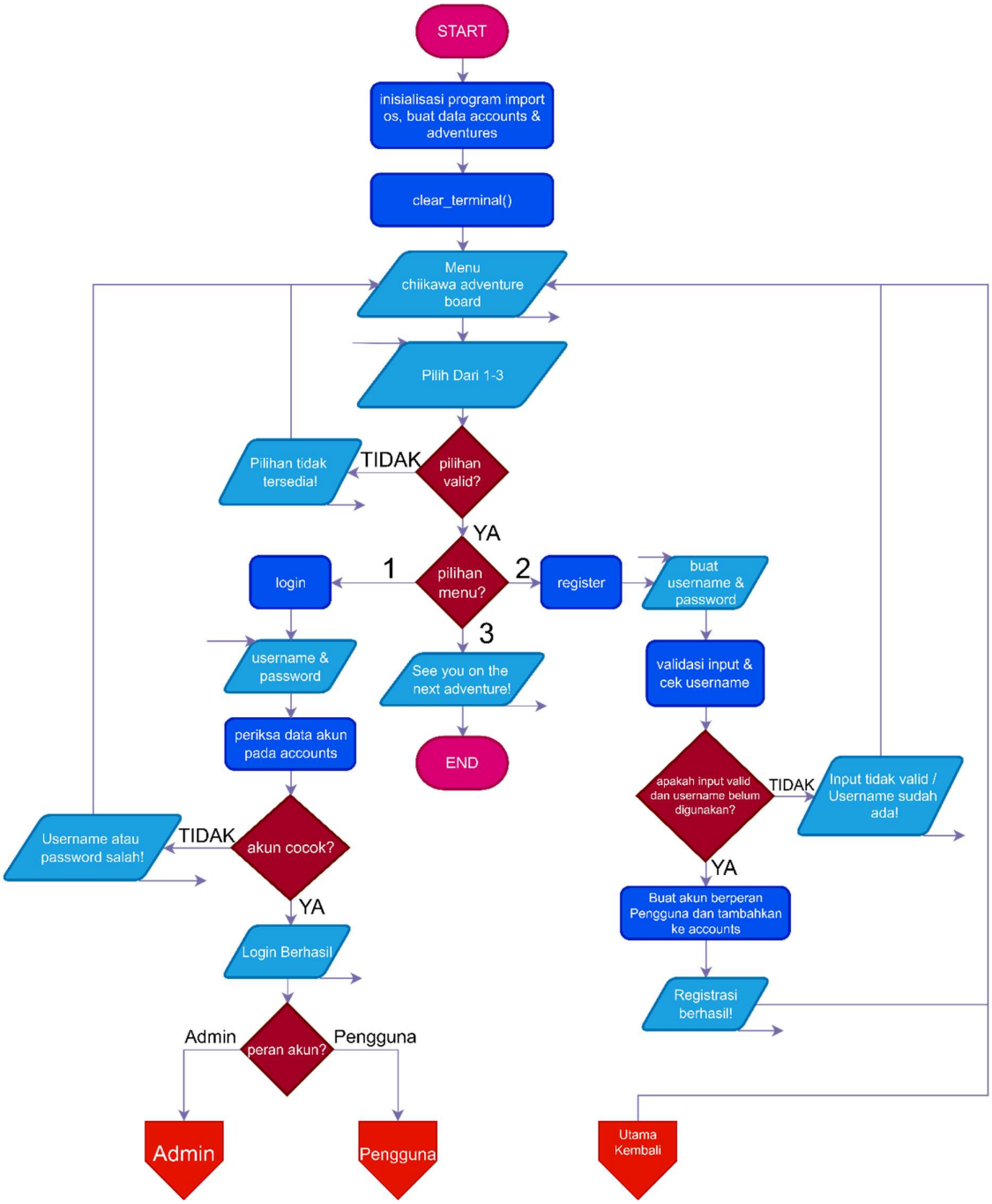
PROGRAM STUDI INFORMATIKA

UNIVERSITAS MULAWARMAN

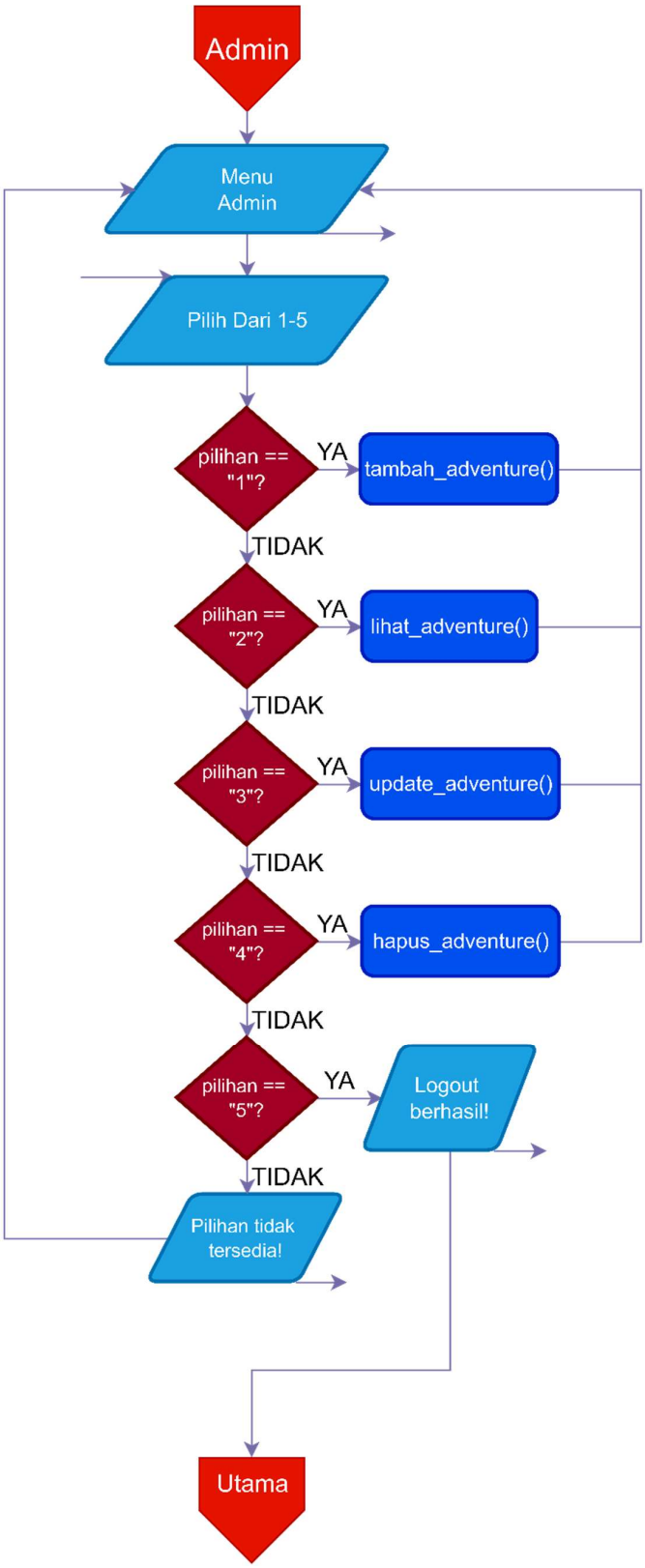
SAMARINDA

2026

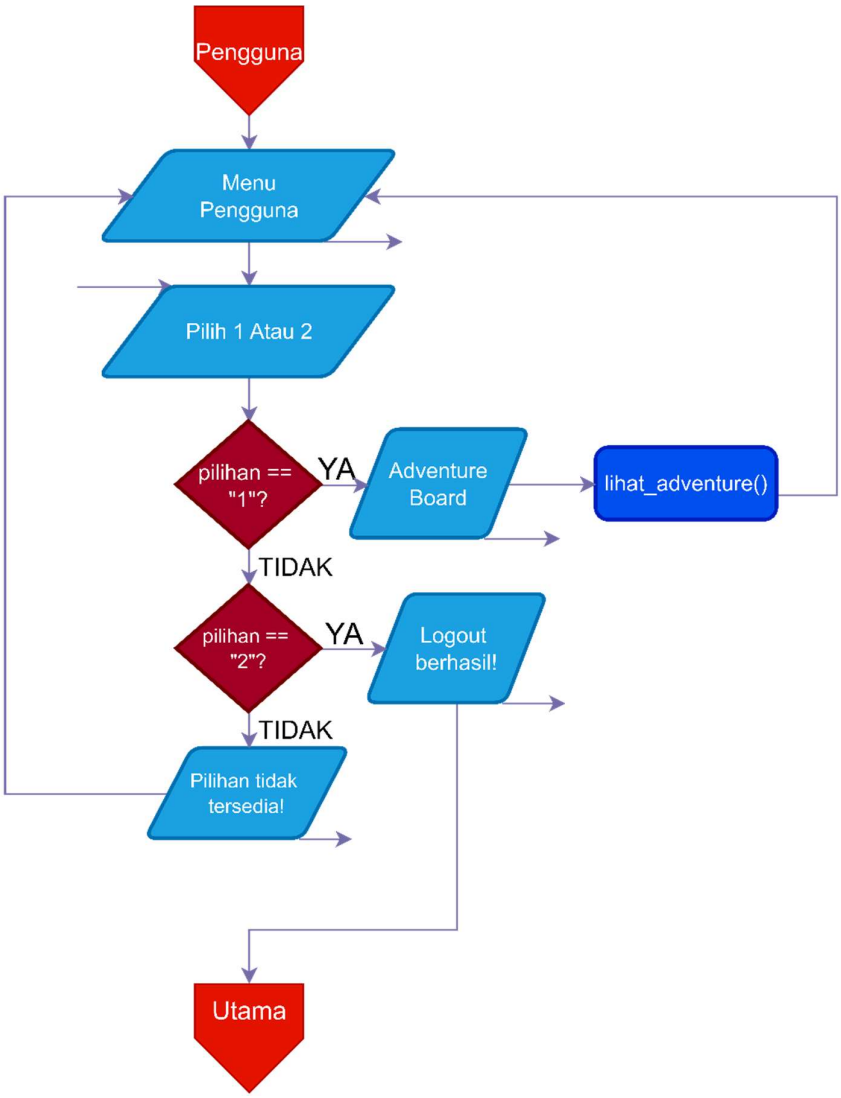
1. Flowchart



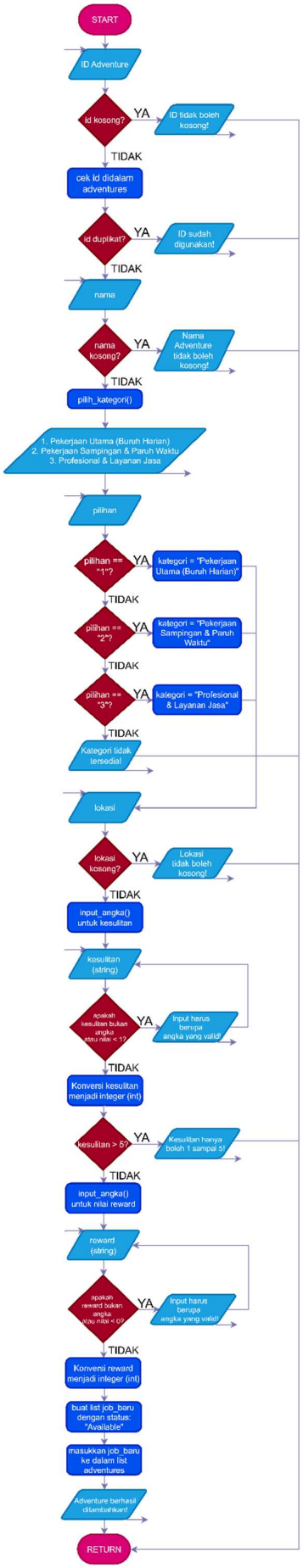
Gambar 1.1 Flowchart Menu Beranda Utama dan Otentikasi Akun Sistem



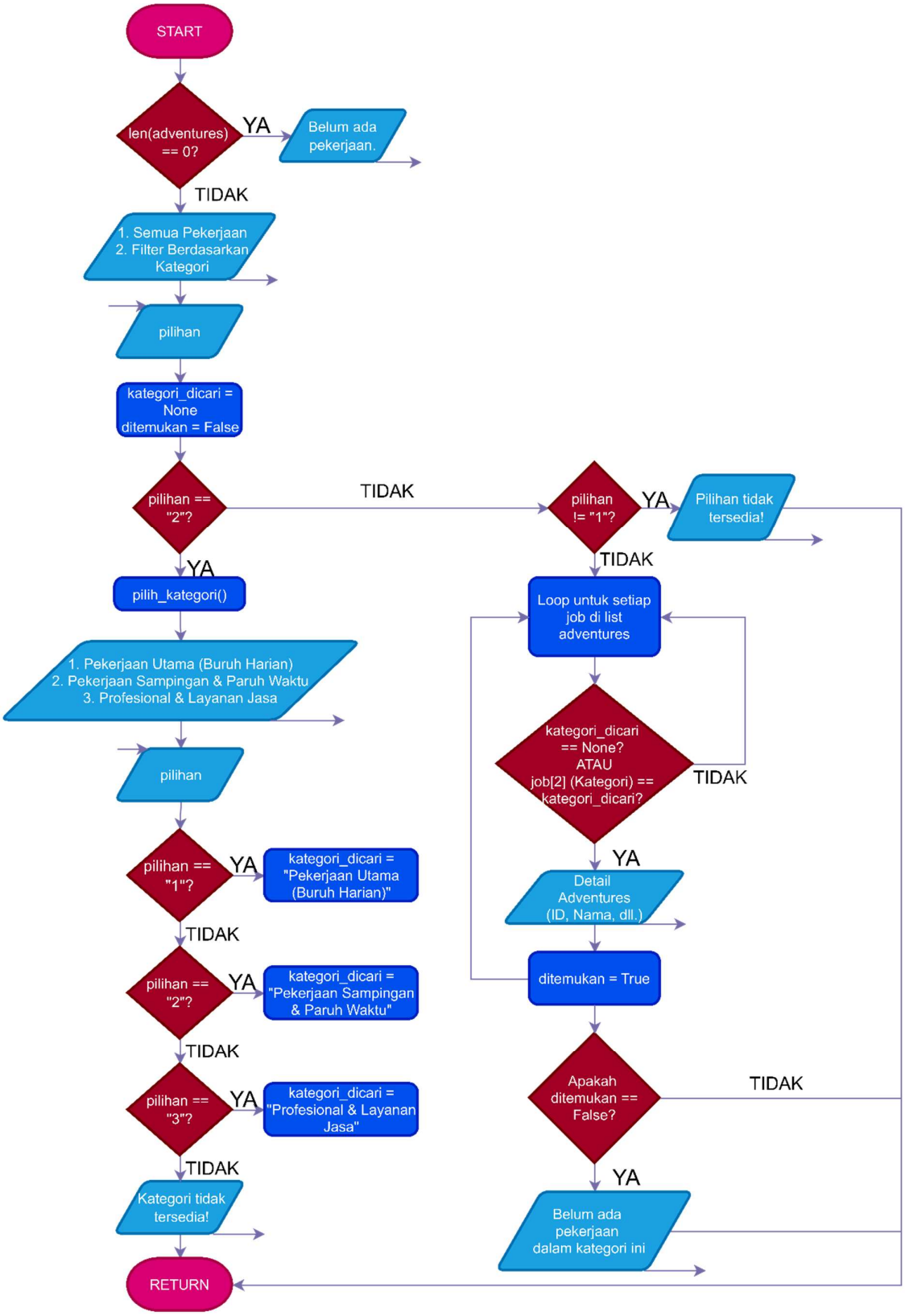
Gambar 1.2 Flowchart Navigasi Menu Interaktif Admin



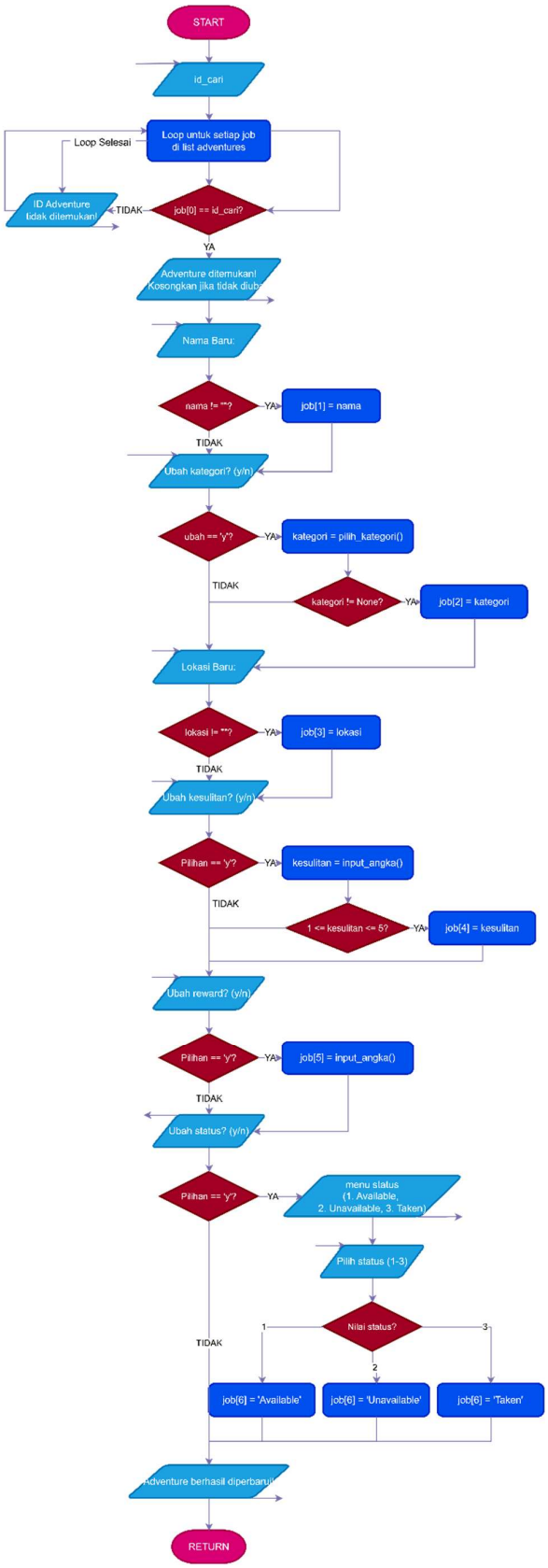
Gambar 1.3 Flowchart Navigasi Menu Interaktif Pengguna



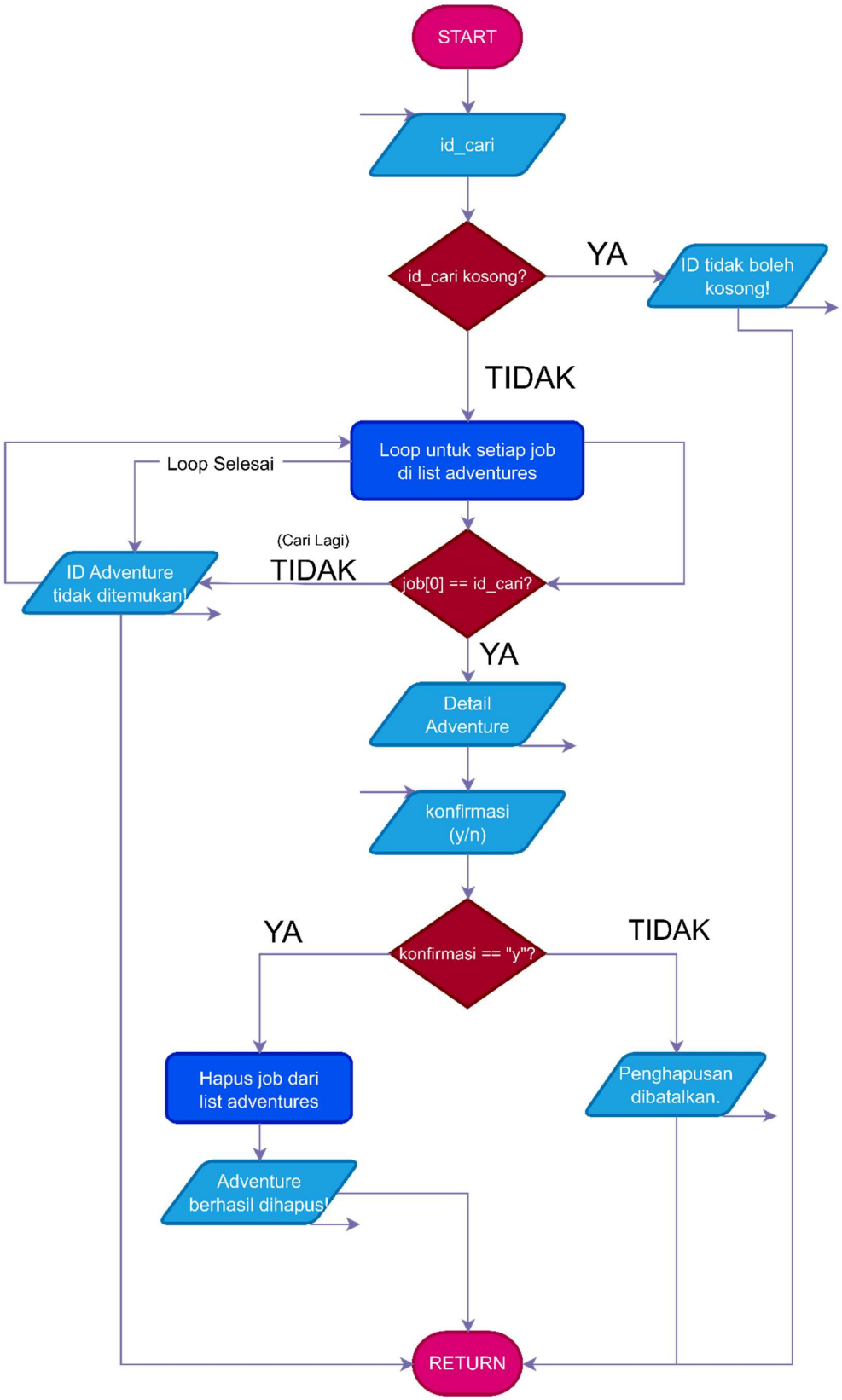
Gambar 1.4 Flowchart Fungsi tambah_adventure() Modul Operasi Penambahan Adventures Baru



Gambar 1.5 Flowchart Fungsi lihat_adventure() Papan Mading Informasi dan Filter Kategori



Gambar 1.6 Flowchart Fungsi update_adventure() Modifikasi Parameter dan Status Misi Petualangan



Gambar 1.7 Flowchart Fungsi hapus_adventure() dengan Konfirmasi Keamanan Berlapis

a) Penjelasan *Flowchart*

Flowchart tersebut menampilkan langkah-langkah operasional untuk aplikasi manajemen misi petualangan digital. Program ini dilengkapi dengan sistem validasi keamanan ganda yang memeriksa ketepatan data *login* akun serta batasan parameter pengisian tingkat kesulitan dan *reward* pekerjaan. Selain memiliki menu navigasi dasbor yang interaktif bagi Admin maupun Pengguna, program ini juga menerapkan penyeleksian filter berjenjang berdasarkan kategori misi petualangan serta otomatis mengeluarkan bukti cetak detail spesifikasi pekerjaan saat misi sukses ditambahkan ke papan mading.

Flowchart Utama (Main)

Dimulai dengan proses Main / Start. Alur langsung mengarah pada blok *Process* inisialisasi awal program untuk mengimpor modul *os* serta menyiapkan struktur data list dua dimensi, yaitu variabel *accounts* dan *adventures*. Selanjutnya, program memanggil fungsi clear_terminal() untuk membersihkan layar terminal sebelum menampilkan halaman beranda utama perbankan petualangan. Alur kemudian memasuki perulangan beranda utama (*looping*) dengan mencetak *Output* banner teks "CHIIKAWA ADVENTURE BOARD" beserta menu pilihan: angka 1 untuk Login, angka 2 untuk Register, dan angka 3 untuk *Exit*. Proses *Input* mengambil nilai dari terminal dan menyimpannya ke dalam variabel pilihan. Alur lalu masuk ke dalam rangkaian struktur kondisi penyeleksian *Decision*:

- Jika pilihan == "3" (YA): Sistem menampilkan *Output string* perpisahan "*See you on the next adventure!*", memutus perulangan, dan bergerak lurus mengakhiri seluruh jalannya diagram menuju END.
- Jika pilihan bukan angka 1, 2, atau 3 (TIDAK pada validasi awal): Alur otomatis diarahkan menuju *Output* peringatan "[Pilihan tidak tersedia!]" dan panah alurnya memutar kembali ke atas beranda utama.
- Jika pilihan == "2" (YA): Program mengarahkan alur masuk ke dalam blok fungsi register(). Di sini pengguna meng*input* data ke terminal. Apabila data kosong atau *string username* yang dimasukkan terdeteksi duplikat (Decision bernilai YA), sistem mengeluarkan *Output error* dan menarik garis kembali ke atas beranda. Namun jika data lolos kualifikasi, data ditambahkan ke dalam list accounts, memicu *Output* "Registrasi berhasil!", lalu ditarik kembali ke atas beranda.
- Jika pilihan == "1" (YA): Program mengarahkan alur masuk ke dalam fungsi login(). Proses *Input* akan mengambil nilai *string username* dan *password* petualang. Proses *Decision* mengecek kecocokan data dengan akun terdaftar di *database*. Jika TIDAK cocok, *Output* "Username atau password salah!" dicetak dan alur memutar kembali ke menu utama beranda. Jika YA (cocok), status sukses login terpenuhi, memicu *Output* teks "Login Berhasil", dan alur dilanjutkan menuju percabangan penentu

hak akses peran (*Decision* peran akun). Jika hak akses yang terdeteksi adalah "Admin", alur melompat masuk menuju modul percabangan interaktif halaman menu_admin(). Sedangkan jika hak akses petualang bernilai "Pengguna", alur dialihkan masuk ke dalam rangkaian perulangan sub-diagram menu_pengguna().

Flowchart Fungsi Tambahan (Sub-Diagram)

- Fungsi tambah_adventure(): Diagram diawali dengan START dan meminta *Input* data id_adventure. Tahap *Decision* mengecek apakah *input* kosong atau duplikat. Jika YA, program memunculkan *Output* kegagalan dan mengakhiri fungsi lewat RETURN. Jika lolos, pengguna menginput data nama, memanggil fungsi penentu kategori, lokasi, serta tingkat kesulitan dan besaran *reward* numerik integer melalui perantara validasi fungsi input_angka(). Setelah seluruh data tervalidasi aman, proses matematika memasukkan data *array* baru dijalankan ke *list adventures* dengan default status "Available", memicu *Output* "Adventure berhasil ditambahkan!", lalu ditutup dengan RETURN.
- Fungsi lihat_adventure(): Diagram dimulai dengan memeriksa kondisi *Decision* len(adventures) == 0. Jika YA, program mencetak *Output* "Belum ada pekerjaan." lalu keluar. Jika TIDAK, sistem memunculkan menu interaktif untuk melihat semua pekerjaan atau memfilternya berdasarkan kategori. Masukan *Input* diseleksi; jika memilih filter kategori, pengguna wajib melalui fungsi percabangan pemilihan kategori terlebih dahulu. Selanjutnya program menjalankan perulangan *Loop* berjenjang untuk menyisir setiap baris data di dalam *list adventures*. Di dalam perulangan ini terdapat *Decision* pencocokan filter kategori. Jika data cocok, *Output* detail isi data petualangan akan dicetak terformat ke layar. Jika perulangan menyisir selesai dan tidak ada satu pun data yang sesuai, sistem memicu *Output* "Belum ada pekerjaan dalam kategori ini." sebelum diagram ditutup rapi dengan RETURN.
- Fungsi update_adventure(): Mengambil data masukan *Input* berupa *string* id_cari. Program memasuki proses perulangan *Loop* menyisir isi list petualangan untuk mencari kesamaan ID. Jika data target tidak ditemukan hingga *loop* selesai, *Output error* "ID Adventure tidak ditemukan!" akan muncul. Jika *Decision* kecocokan bernilai YA, sistem memberikan instruksi *input* pembaruan. Pengguna dapat mengosongkan *input* nama atau lokasi jika tidak ingin mengubahnya. Untuk perubahan kategori, kesulitan, *reward*, hingga pembaruan status kerja petualangan (Available, Unavailable, Taken), data diseleksi secara ketat menggunakan fungsi penyeleksi bertingkat. Setelah eksekusi pergantian nilai variabel sukses, *Output* "Adventure berhasil diperbarui!" dicetak dan alur diputus keluar menuju RETURN.

- Fungsi hapus_adventure(): Meminta *Input* data target id_cari. Jalur penyisiran *Loop* dijalankan. Jika ID ditemukan, sistem tidak langsung menghapus data melainkan masuk ke gerbang keamanan konfirmasi akhir. Proses *Input* meminta karakter (y/n). Jika *Decision* berharga YA ("y"), proses penghapusan data *array* internal .remove(job) dieksekusi, memicu *Output* sukses "Adventure berhasil dihapus!". Jika masukan bernilai TIDAK, sistem mencetak *Output* pembatalan "Penghapusan dibatalkan.", dan alur diarahkan lurus selesai menuju terminal RETURN.

2. Deskripsi Singkat Program

Program ini merupakan sistem aplikasi papan informasi petualangan digital bertajuk **"Chiikawa Adventure Board"** yang dirancang untuk memproses manajemen data penawaran pekerjaan (misi petualangan) berbasis *multi-user* (Admin dan Pengguna). Program ini memiliki sistem keamanan proteksi akses, di mana fitur krusial manipulasi *database* diletakkan di balik gerbang validasi khusus peran akun. Tak hanya itu, program ini juga dilengkapi dengan fungsi validasi numerik bawaan untuk menangani kesalahan ketik (*error handling* data non-digit), menu *dashboard* interaktif yang dinamis, sistem filter data pencarian misi berjenjang berdasarkan kategori, serta fitur konfirmasi keamanan berlapis sebelum melakukan operasi penghapusan berkas data permanen di terminal.

a) Alur Kerja Program

Alur kerja program diawali dengan eksekusi fungsi pembersih halaman terminal dan langsung menampilkan menu beranda awal sistem yang meminta pengguna memilih opsi masuk aplikasi. Melalui menu registrasi, sistem akan memvalidasi agar *string* data username baru tidak boleh kosong serta wajib bersifat unik (tidak boleh duplikat dengan nama petualang yang sudah terdaftar di *database* list accounts). Pada menu masuk, identitas i diperiksa silang. Jika data otentikasi valid, program membaca hak peran akses petualang. Apabila masuk sebagai peran "Admin", sistem akan membuka modul menu khusus yang mengizinkan operasi CRUD penuh (membuat misi baru, melihat seluruh papan mading, menyunting detail limit kesulitan, status, dan hadiah, serta menghapus data penawaran misi). Sebaliknya, jika *login* sebagai peran "Pengguna", sistem hanya akan membuka akses modul mading baca-saja (*read-only*) untuk mengamati ketersediaan papan misi petualangan.

Dalam pengerjaan operasional data misi petualangan, sistem menerapkan standarisasi tipe data dan batasan nilai yang ketat. Pengisian tingkat kesulitan misi dibatasi wajib berada di dalam rentang angka integer 1 sampai 5. Sementara nilai total *reward* bonus hadiah diatur minimal bernilai angka 0. Setiap kali terjadi kesalahan pengisian *input* angka numerik atau limit jangkauan dilanggar, fungsi asisten input_angka() secara adaptif menolak data, memunculkan notifikasi kesalahan teks error terformat, dan memaksa pengguna mengisi ulang hingga valid. Papan informasi pekerjaan juga menyediakan opsi klasifikasi filter agar pengguna dapat menyaring tampilan berdasarkan tiga kategori utama pekerjaan (Pekerjaan Utama, Pekerjaan Sampingan, atau Profesional) secara rapi. Sesi navigasi aplikasi

akan terus berputar secara interaktif di dalam terminal menggunakan kendali perulangan tak terbatas *while True*, hingga pengguna secara sadar memilih menu keluar (*Exit*) yang akan memicu pencetakan pesan penutup aplikasi dan memutus rantai proses eksekusi kode program secara aman dan bersih.

3. Source Code

A. Fitur Kontrol Validasi Input Angka dan Error Handling

Fitur ini digunakan untuk memastikan data masukan dari terminal wajib berupa digit angka murni serta memenuhi standar batas nilai minimum untuk mencegah terjadinya *crash* sistem akibat kesalahan tipe data.

Source Code:

```
def input_angka(pesan, minimum):
    while True:
        nilai = input(pesan)

        if nilai.isdigit() and int(nilai) >= minimum:
            return int(nilai)

        print("Input harus berupa angka yang valid!")
```

B. Fitur Proteksi Duplikasi Identitas Akun Baru

Fitur ini disematkan di dalam sistem registrasi untuk memvalidasi ketersediaan *string* masukan agar tidak terjadi penumpukan nama pengguna (username) yang sama di dalam database array.

Source Code:

```
for akun in accounts:
    if akun[0] == username:
       print("Username sudah digunakan!")
       return
```

C. Fitur Otentikasi dan Identifikasi Hak Akses Multi-User

Fitur ini bertindak sebagai gerbang pengaman untuk mencocokkan kredensial *login* pengguna sekaligus membaca hak peran akses yang terikat pada akun tersebut demi menentukan tampilan menu yang sesuai.

Source Code:

```
for akun in accounts:
    if akun[0] == username and akun[1] == password:
       print(f"Login berhasil! Selamat datang {username}")
       return akun
```

D. Fitur Seleksi Pembatasan Nilai Aturan Batasan Jangkauan Adventure

Fitur ini diterapkan di dalam pengerjaan penambahan adventure baru untuk mengontrol agar parameter tingkat tingkat kesulitan pengerjaan adventure wajib patuh pada batas aturan rentang skala 1 hingga 5.

Source Code:

```
kesulitan = input_angka("Tingkat Kesulitan (1-5): ", 1)

if kesulitan > 5:
   print("Kesulitan hanya boleh 1 sampai 5!")
   return
```

E. Fitur Penyaringan dan Tampilan Filter Informasi Berjenjang

Fitur ini berfungsi memilah susunan baris mading petualangan secara kondisional, sehingga terminal dapat menyajikan ringkasan data yang bersifat global maupun yang telah terfilter berdasarkan kecocokan *string* kategori tertentu.

Source Code:

```
for job in adventures:
    if kategori_dicari is None or job[2] == kategori_dicari:
        print("-----------------------------------")
        print("ID         :", job[0])
        print("Pekerjaan  :", job[1])
        print("Kategori   :", job[2])
        print("Lokasi     :", job[3])
        print("Kesulitan  :", job[4])
        print("Reward     :", job[5])
        print("Status     :", job[6])

        ditemukan = True
```

F. Fitur Sunting Parsial Nilai Array Database Terpilih

Fitur ini digunakan untuk mengubah data di dalam baris list secara dinamis, di mana sistem memberikan fleksibilitas agar *user* cukup menekan tombol *Enter* (mengosongkan *input*) jika tidak berniat mengubah nilai lama data tersebut.

Source Code:

```
nama = input("Nama Baru: ")
if nama != "":
    job[1] = nama
```

G. Fitur Pengaman Berlapis Operasi Hapus Data

Fitur ini bertindak sebagai fungsi konfirmasi pengaman darurat sebelum program mengeksekusi perintah penghapusan data secara permanen menggunakan *method* bawaan python .remove() guna meminimalkan risiko kehilangan data akibat ketidaksengajaan.

Source Code:

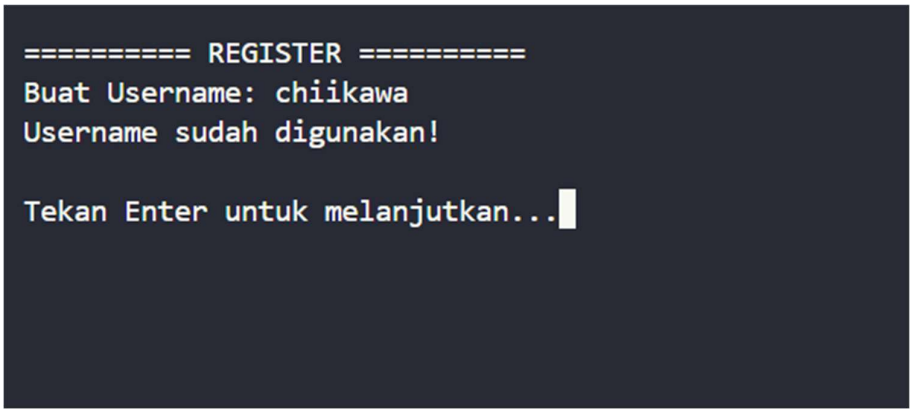
```
konfirmasi = input("Yakin Ingin Menghapus Pekerjaan Ini? (y/n): ")

if konfirmasi == "y":
   adventures.remove(job)
   print("Adventure berhasil dihapus!")
else:
   print("Penghapusan dibatalkan.")
```

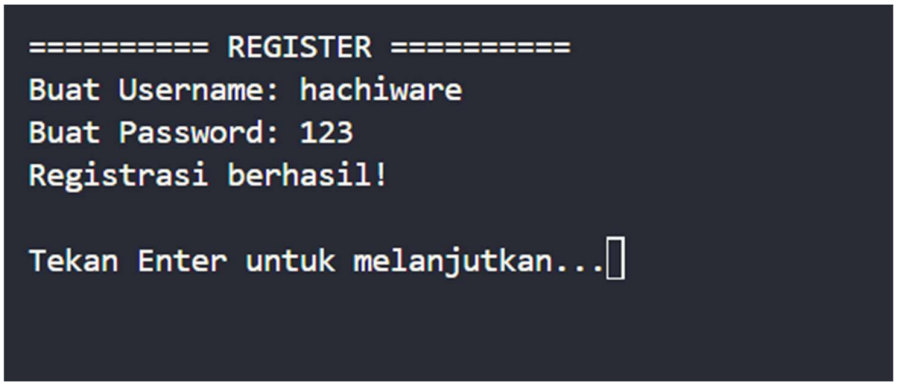
4. Hasil Output



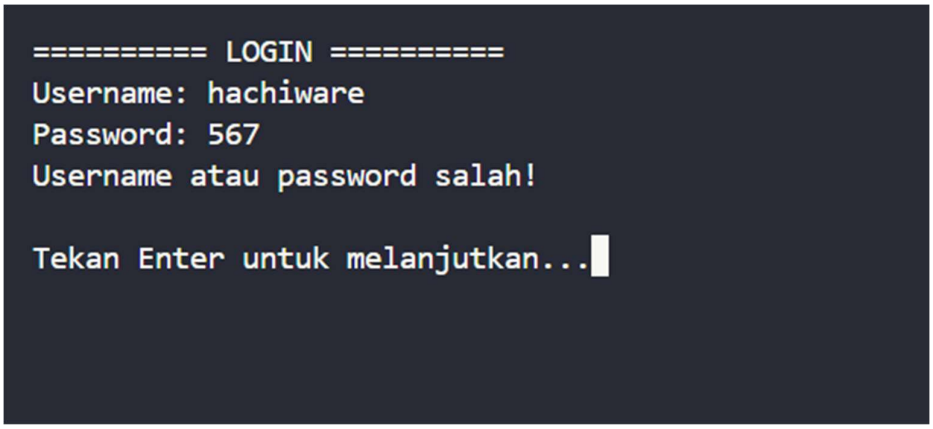
Gambar 4.1 Output Tampilan Menu Utama Beranda *Chiikawa Adventure Board* dan Uji Coba Pilihan 3



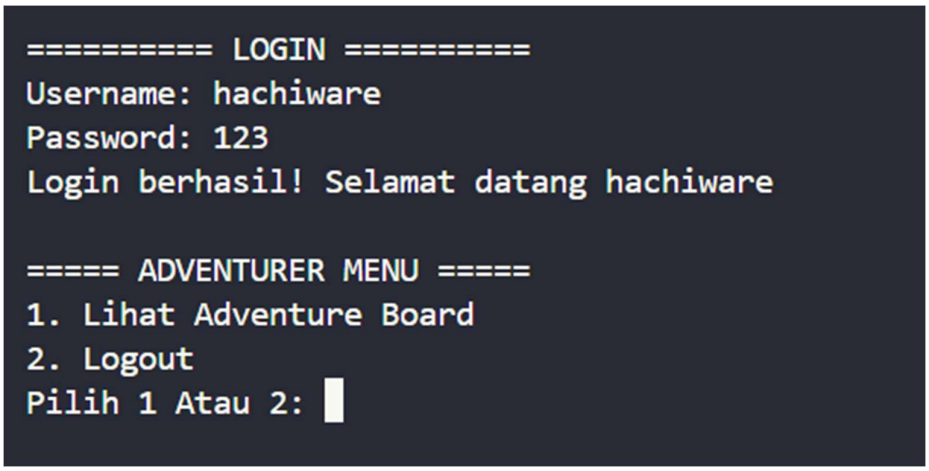
Gambar 4.2 Output Peringatan Sistem Jika *Username* Terdeteksi Duplikat



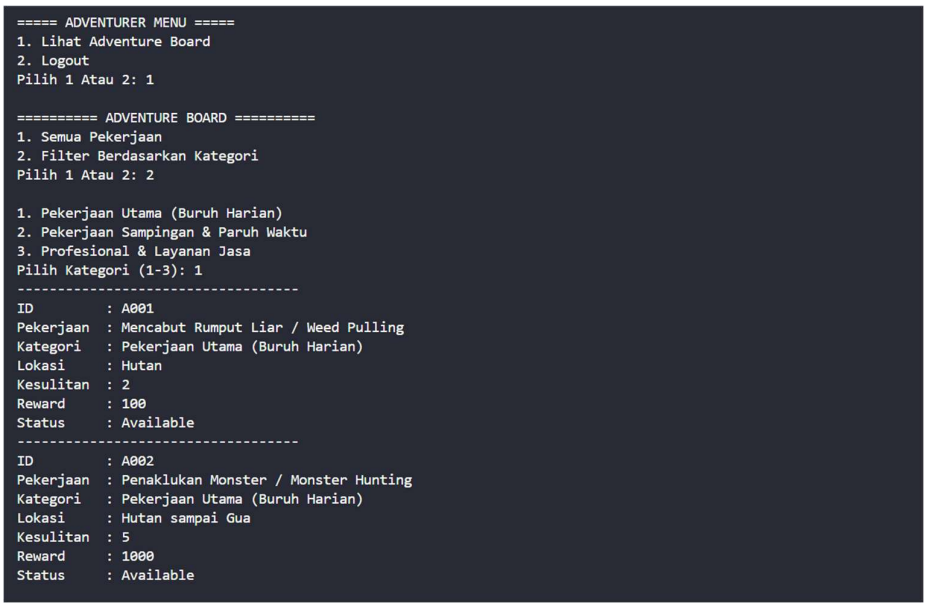
Gambar 4.3 Output Keberhasilan Registrasi Akun Petualang Baru



Gambar 4.4 Output Peringatan Sistem Pada *Login* Jika *Username* atau *Password* Salah



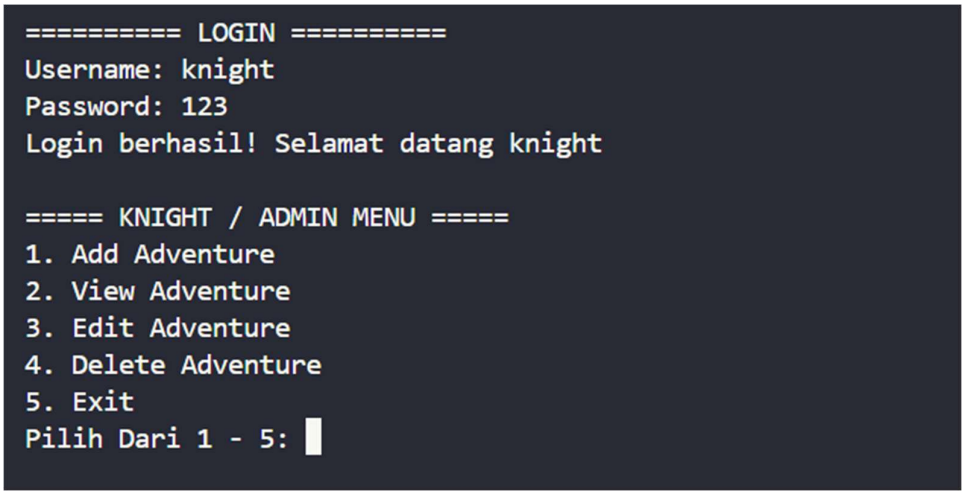
Gambar 4.5 Output Keberhasilan Login dan Tampilan Dasbor Untuk Peran Pengguna



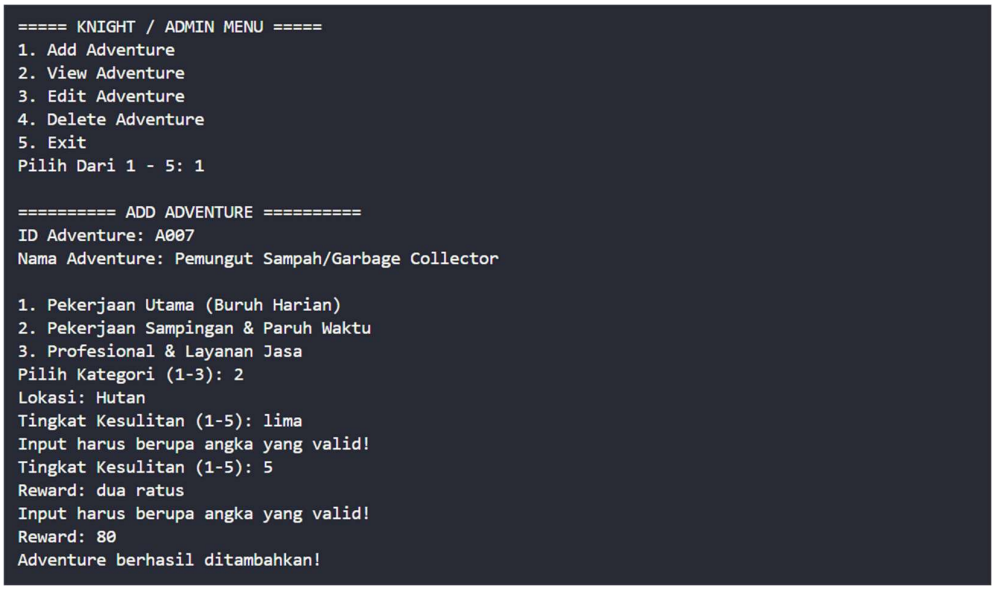
Gambar 4.6 Output Hasil Penyaringan (*Filtering*) Tampilan Papan Mading Berdasarkan Kategori Spesifik

```
========== ADVENTURE BOARD ==========
1. Semua Pekerjaan
2. Filter Berdasarkan Kategori
Pilih 1 Atau 2: 1
-----------------------------------
ID         : A001
Pekerjaan  : Mencabut Rumput Liar / Weed Pulling
Kategori   : Pekerjaan Utama (Buruh Harian)
Lokasi     : Hutan
Kesulitan  : 2
Reward     : 100
Status     : Available
-----------------------------------
ID         : A002
Pekerjaan  : Penaklukan Monster / Monster Hunting
Kategori   : Pekerjaan Utama (Buruh Harian)
Lokasi     : Hutan sampai Gua
Kesulitan  : 5
Reward     : 1000
Status     : Available
-----------------------------------
ID         : A003
Pekerjaan  : Menempel Stiker / Stickering Lemons
Kategori   : Pekerjaan Sampingan & Paruh Waktu
Lokasi     : Pabrik
Kesulitan  : 1
Reward     : 80
Status     : Available
-----------------------------------
ID         : A004
Pekerjaan  : Memetik Jamur / Mushroom Picking
Kategori   : Pekerjaan Sampingan & Paruh Waktu
Lokasi     : Hutan
Kesulitan  : 3
Reward     : 100
Status     : Available
-----------------------------------
ID         : A005
Pekerjaan  : Pegawai Kedai Ramen
Kategori   : Pekerjaan Profesional & Layanan Jasa
Lokasi     : Kedai Ramen
Kesulitan  : 4
Reward     : 800
Status     : Available
-----------------------------------
ID         : A006
Pekerjaan  : Pengatur Lalu Lintas / Polisi
Kategori   : Pekerjaan Profesional & Layanan Jasa
Lokasi     : Jalan Raya
Kesulitan  : 4
Reward     : 900
Status     : Available
```

Gambar 4.7 Output Tampilan Menyeluruh Daftar Informasi Isi Papan Mading Petualangan



Gambar 4.8 Output Keberhasilan Login dan Tampilan Dasbor Untuk Peran Admin



Gambar 4.9 Output Pengisian Formulir dan Keberhasilan Penambahan Data Misi Petualangan Baru Serta Penolakan Sistem Jika Input Nilai Numerik Berupa Karakter/Huruf

```
========== ADD ADVENTURE ==========
ID Adventure: A008
Nama Adventure: Barista

1. Pekerjaan Utama (Buruh Harian)
2. Pekerjaan Sampingan & Paruh Waktu
3. Profesional & Layanan Jasa
Pilih Kategori (1-3): 3
Lokasi: Kafe
Tingkat Kesulitan (1-5): 6
Kesulitan hanya boleh 1 sampai 5!

===== KNIGHT / ADMIN MENU =====
1. Add Adventure
2. View Adventure
3. Edit Adventure
4. Delete Adventure
5. Exit
Pilih Dari 1 - 5:
```

Gambar 4.10 Output Notifikasi Kesalahan Jika Parameter Skala Kesulitan Melebihi Batas Aturan (1-5)

```
===== KNIGHT / ADMIN MENU =====
1. Add Adventure
2. View Adventure
3. Edit Adventure
4. Delete Adventure
5. Exit
Pilih Dari 1 - 5: 6

Pilihan tidak tersedia!
```

Gambar 4.11 Output Peringatan Sistem Jika Pilihan Angka Menu Dasbor Tidak Valid

```
===== KNIGHT / ADMIN MENU =====
1. Add Adventure
2. View Adventure
3. Edit Adventure
4. Delete Adventure
5. Exit
Pilih Dari 1 - 5: 3

========== UPDATE ADVENTURE ==========
Masukkan ID Adventure: A007
Adventure ditemukan!
Kosongkan nama atau lokasi jika tidak ingin mengubahnya.
Nama Baru:
Ubah kategori? (y/n)
Pilihan: n
Lokasi Baru:
Ubah kesulitan (y/n)
Pilihan: y
Tingkat Kesulitan (1-5): 2
Ubah reward (y/n)
Pilihan: y
Reward Baru: 85
Ubah status? (y/n)
Pilihan: n
Adventure berhasil diperbarui!

===== KNIGHT / ADMIN MENU =====
1. Add Adventure
2. View Adventure
3. Edit Adventure
4. Delete Adventure
5. Exit
Pilih Dari 1 - 5:
```

Gambar 4.12 Output Proses Pencarian ID dan Keberhasilan Penyuntingan (*Update*) Parameter Misi

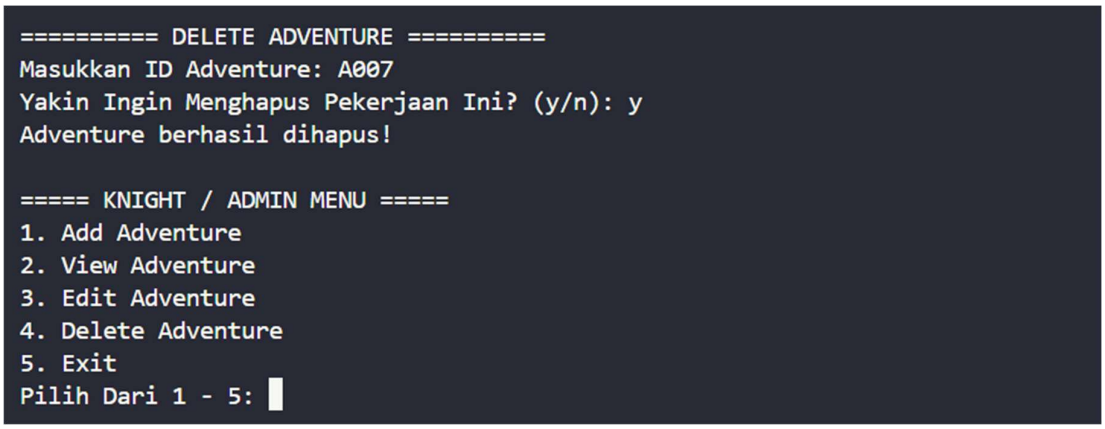
```
ID          : A007
Pekerjaan   : Pemungut Sampah/Garbage Collector
Kategori    : Pekerjaan Sampingan & Paruh Waktu
Lokasi      : Hutan
Kesulitan   : 2
Reward      : 85
Status      : Available
```

Gambar 4.13 Hasil Dari *Update* Parameter Misi

```
========== DELETE ADVENTURE ==========
Masukkan ID Adventure: A007
Yakin Ingin Menghapus Pekerjaan Ini? (y/n): n
Penghapusan dibatalkan.

===== KNIGHT / ADMIN MENU =====
1. Add Adventure
2. View Adventure
3. Edit Adventure
4. Delete Adventure
5. Exit
Pilih Dari 1 - 5:
```

Gambar 4.14 Output Gerbang Konfirmasi Akhir dan Pembatalan Penghapusan (*Delete*) Data Misi

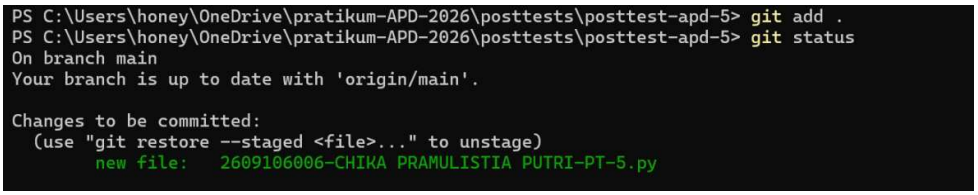


Gambar 4.15 Output Gerbang Konfirmasi Akhir dan Keberhasilan Penghapusan (*Delete*) Data Misi

5. Langkah-langkah Git

5. 1 Git Add

Fitur *git add* digunakan untuk memindahkan berkas kode program dari direktori kerja (*Working Directory*) menuju area penyiapan (*Staging Area*).

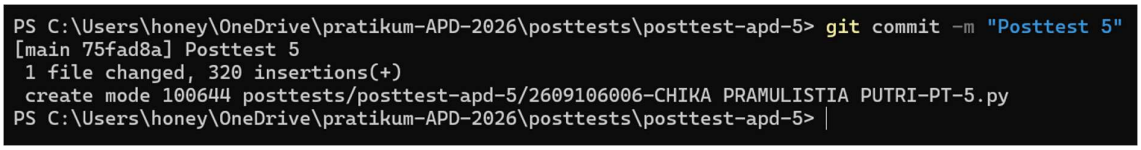


Gambar 5.1 Hasil Pemindahan Berkas ke *Staging Area* Menggunakan *git add*

Proses diawali dengan membuka terminal sistem di dalam folder repositori lokal pengerjaan tugas. Pengguna langsung mengeksekusi perintah *git add .* untuk memasukkan berkas program baru ke dalam antrean pelacakan Git secara massal. Setelah itu, perintah *git status* dijalankan untuk memastikan berkas telah berhasil dipindahkan. Proses ini dinyatakan sukses ketika terminal menampilkan nama berkas dengan teks berwarna hijau (*Changes to be committed*), menandakan berkas telah aman berada di *Staging Area* dan siap dikunci ke tahap selanjutnya.

5.2 Git Commit

Fitur *git commit* digunakan untuk menyimpan dan mengunci seluruh riwayat perubahan berkas dari *Staging Area* secara permanen ke dalam basis data repositori lokal.

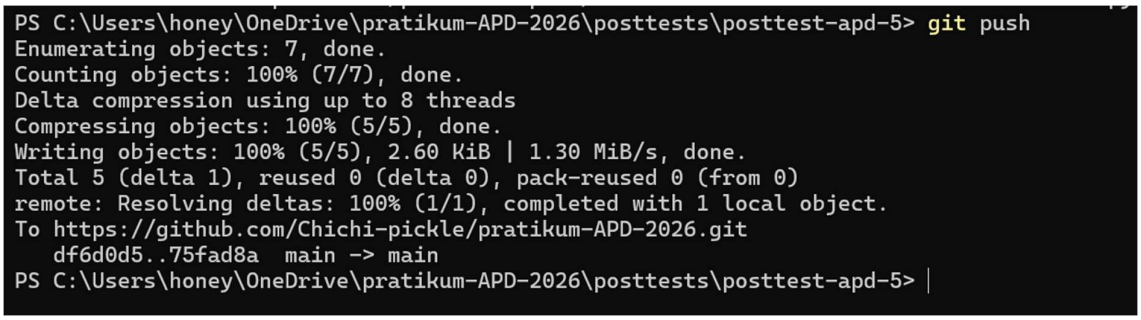


Gambar 5.2 Hasil Penguncian Riwayat Perubahan Melalui *git commit*

Setelah berkas dipastikan berwarna hijau pada pengecekan status, perintah *git commit -m "Posttest 5"* dijalankan. Flag *-m* berfungsi sebagai label dokumentasi untuk mencatat jenis perubahan yang dilakukan. Setelah menekan *Enter*, Git akan mengunci riwayat perubahan tersebut dan menampilkan ringkasan jumlah baris kode yang berhasil disimpan.

5.3 Git Push

Fitur git push digunakan untuk mengunggah dan menyelaraskan seluruh catatan riwayat commit dari repositori lokal ke dalam repositori daring di GitHub.



Gambar 5.3 Hasil Pengunggahan Berkas ke *GitHub* Melalui *git push*

Langkah terakhir ini berfungsi untuk mencadangkan berkas kode program secara daring ke platform *GitHub*. Pengunggahan dilakukan dengan mengeksekusi perintah *git push*. Terminal akan memproses pengiriman data ke peladen awan. Proses sinkronisasi ini dinyatakan berhasil sepenuhnya setelah terminal menampilkan log transfer data mencapai status 100% (*done*).